ClassCord Server

Service de tchat multi-clients en Python

Guide de déploiement, de sécurisation et d'exploitation sur Debian

Auteur

Omar Talibi
BTS SIO SISR
IRIS Mediaschool Nice

Stack technique

Python 3 — port TCP/12345
systemd + Docker
ufw + fail2ban

Périmètre

1 VM Debian
5 jours (16 au 20 juin 2025)
8 captures d'écran

Sommaire

01 Contexte et objectifs
02 Architecture
03 Phase 1 — Mise en route et réseau
04 Phase 2 — Service systemd
05 Phase 3 — Conteneurisation Docker
06 Phase 4 — Sécurité : journaux, fail2ban, pare-feu
07 Phase 5 — Sauvegarde automatique
08 Phase 6 — Évolutions fonctionnelles
09 Phase 7 — Finalisation
10 Preuves relevées
11 Difficultés rencontrées
12 Limites et pistes d'amélioration
13 Compétences BTS SIO mobilisées
14 Sources

1 Contexte et objectifs

ClassCord est un serveur de tchat multi-clients écrit en Python (`server_classcord.py`, port TCP/12345). Ses clients Java sont développés par les étudiants SLAM. Ce projet de semaine intensive (16 au 20 juin 2025) confie aux étudiants SISR le déploiement, la sécurisation, la conteneurisation et l'exploitation du serveur, avec une documentation exploitable par un tiers.

Le code de départ vient du dépôt `AstrowareConception/classcord-server`, cloné le jour 2. Le travail et la documentation sont versionnés dans le dépôt `github.com/walo2428/classcord_server`, avec un journal de bord par jour.

> **Mise en situation**
> Mettre à disposition des étudiants un service de communication interne **fiable, documenté et exploitable** : joignable depuis les postes SLAM, démarré automatiquement, protégé contre les connexions abusives et sauvegardé.

Ce que le projet couvre

- VM Debian sous VirtualBox, utilisateur dédié et accès réseau pour les clients SLAM
- Service systemd avec démarrage automatique et utilisateur système `classcord`
- Image Docker et `docker-compose` pour un déploiement reproductible
- Journalisation, pare-feu ufw, fail2ban et sauvegarde planifiée par cron
- Évolutions du serveur : canaux, messages système, export CSV/JSON, interface d'administration
- Documentation technique et versionnement Git avec un push par jour

2 Architecture

Un seul serveur, la VM Debian de l'étudiant SISR. Les clients SLAM se connectent à l'adresse de l'hôte physique, qui relaie le port 12345 vers la VM.

Élément	Support	Rôle	Accès
VM ClassCord (VirtualBox)	Debian	Serveur de tchat Python Écoute sur 0.0.0.0:12345	TCP/12345
Poste SISR (hôte physique)	Poste de travail	Relais vers la VM (redirection NAT)	IP de l'hôte : 10.0.108.42
Postes SLAM	Client Java	Clients du tchat	IP de l'hôte, port 12345

```
Poste SLAM  ->  (redirection NAT)  ->  Poste SISR  ->  VM Debian  ->  port 12345
```

Configuration réseau

Étape	Mode réseau	Adresse
Jour 1	Passage du NAT au **mode pont (bridge)** pour obtenir une vraie IP locale	10.0.108.113/24 (VM)
Jours 2 à 5	Accès des clients SLAM via l'hôte (documentation de connexion)	10.0.108.42:12345 (hôte)

> **À retenir**
> Le mode réseau a évolué : le jour 1, la VM passe en pont pour sortir de l'IP partagée du NAT. La documentation de connexion finale décrit l'accès via l'IP de l'hôte. Il faut toujours préciser quelle IP (VM ou hôte) est donnée aux clients.

3 Phase 1 — Mise en route et réseau

Lundi 16 juin 2025. Préparation de la machine et premier test du serveur.

Environnement

- VM Debian sous VirtualBox, création de l'utilisateur `projet`
- Ajout de `projet` au groupe `sudo` avec `visudo`
- Réseau : passage du NAT au mode pont, adresse relevée avec `ip a`

Premier lancement

```
ip a                              # inet 10.0.108.113/24
python3 server_classcord.py       # écoute sur 0.0.0.0, port 12345
telnet <IP_VM> 12345              # test de connexion depuis une autre machine : OK
sudo ufw status                   # inactive : aucun port filtré
```

Le serveur écoute bien sur toutes les interfaces et répond à une connexion depuis une autre machine. Le pare-feu est encore inactif : il sera configuré le jour suivant.

4 Phase 2 — Service systemd

Mardi 17 juin 2025. Le serveur doit démarrer seul et tourner sous un compte sans privilèges.

Ouverture du port et vérification

```
sudo ufw allow 12345/tcp
ss -tulpn | grep 12345            # le port est bien en écoute (TCP)
```

Utilisateur système dédié

```
sudo useradd -m classcord
sudo passwd classcord
```

Fichier de service

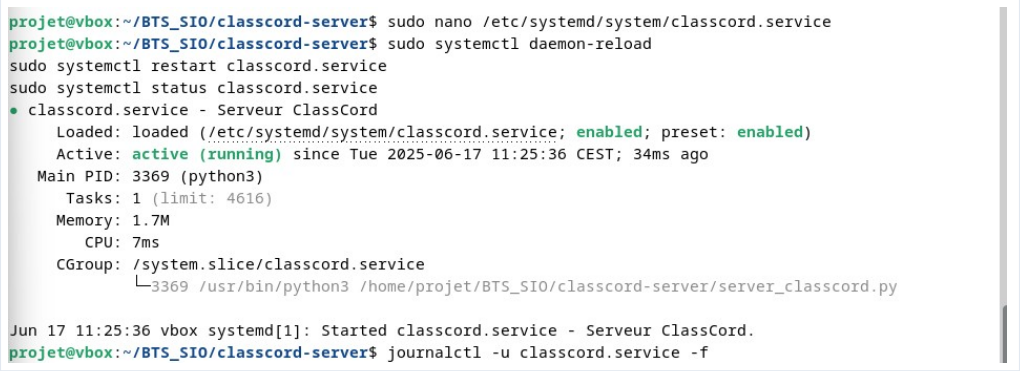
`/etc/systemd/system/classcord.service` :

```
[Unit]
Description=Serveur ClassCord
After=network.target

[Service]
User=classcord
WorkingDirectory=/home/classcord/classcord-server
ExecStart=/usr/bin/python3 /home/classcord/classcord-server/server_classcord.py
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now classcord.service
sudo systemctl status classcord.service
journalctl -u classcord.service -f
```



Capture 01 — Service classcord actif (*active (running)*) et activé au démarrage

> **Remarque**
> Sur la capture, le service lance le dépôt cloné dans `/home/projet/BTS_SIO/classcord-server` (copie de travail). Le fichier ci-dessus est la version documentée avec l'utilisateur `classcord`.

5 Phase 3 — Conteneurisation Docker

Mardi 17 et mercredi 18 juin 2025. Docker apporte un environnement reproductible : mêmes dépendances et même lancement sur n'importe quel hôte.

Pourquoi Docker ?

- **Portabilité** et reproductibilité de l'environnement serveur
- **Isolation** des dépendances et du processus de lancement
- **Déploiement simplifié** avec `docker run` ou `docker compose`

Dockerfile

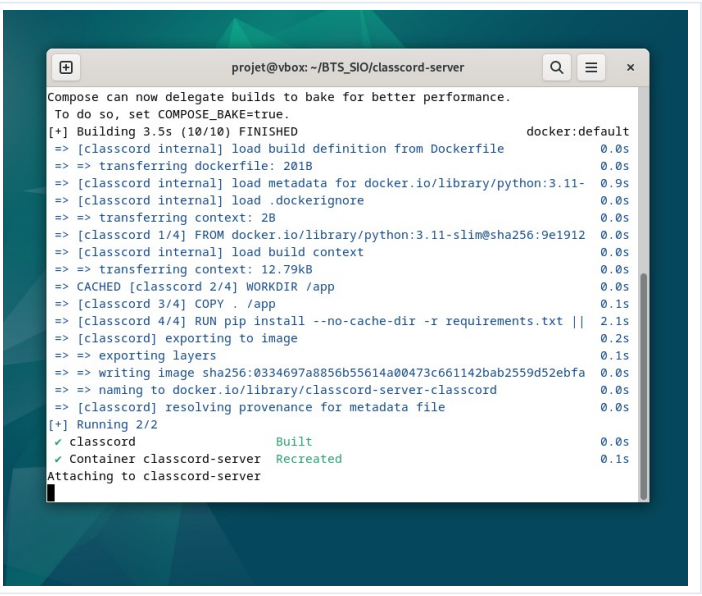
```
FROM python:3.11-slim
WORKDIR /app
COPY . /app
RUN pip install --no-cache-dir -r requirements.txt || true
EXPOSE 12345
CMD ["python", "server_classcord.py"]
```

docker-compose.yml

```
services:
  classcord:
    build: .
    container_name: classcord-server
    ports:
      - "12345:12345"
    restart: unless-stopped
    networks:
      - classcord-net

networks:
  classcord-net:
    driver: bridge
```

```
docker build -t classcord-server .
docker run -it --rm -p 12345:12345 classcord-server
docker compose up -d && docker compose ps
```



Capture 02 — Image construite et conteneur recréé avec docker compose

Redémarrage automatique
`restart: unless-stopped` relance le conteneur après un redémarrage du serveur ou un arrêt inattendu.

6 Phase 4 — Sécurité : journaux, fail2ban, pare-feu

Mercredi 18 juin 2025. Trois protections complémentaires : tracer ce qui se passe, bloquer les comportements anormaux, limiter les ports ouverts.

Journalisation

Le serveur utilise le module `logging` de Python : horodatage, niveau et message, écrits dans un fichier de log et sur la console. Chaque connexion, chaque message reçu et chaque erreur sont tracés.

```
2025-06-18 14:58:12,398 - INFO - Utilisateurs chargés : []
2025-06-18 14:58:12,398 - INFO - Serveur en écoute sur 0.0.0.0:12345
2025-06-19 14:50:33,827 - INFO - Nouvelle connexion depuis ('127.0.0.1', 40474)
```

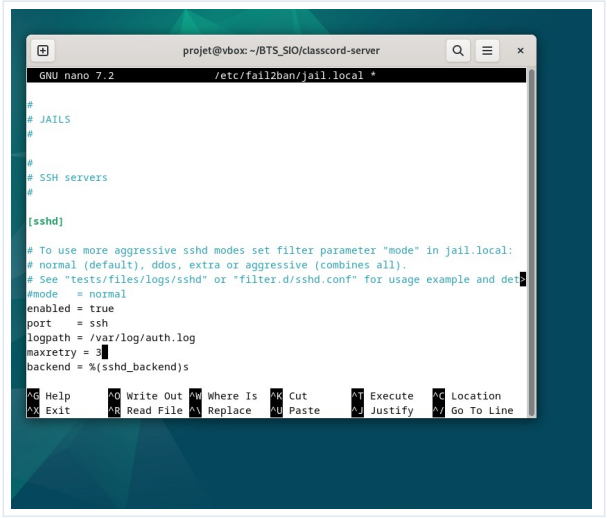
fail2ban

Installation avec `sudo apt install fail2ban`, puis ajout d'une jail dédiée dans `/etc/fail2ban/jail.local` et d'un filtre personnalisé.

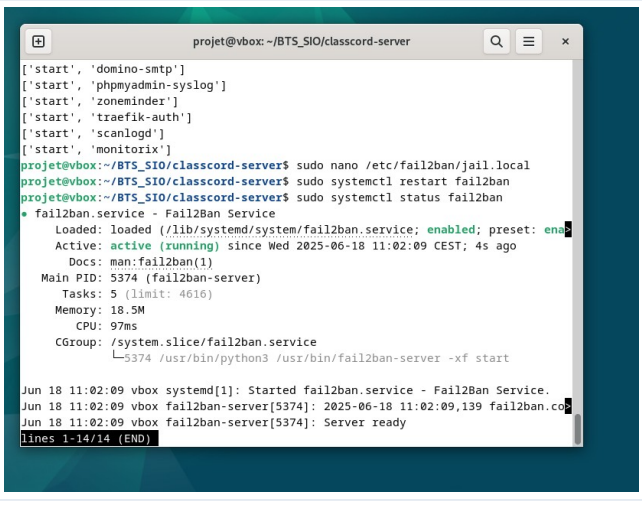
```
# /etc/fail2ban/jail.local
[classcord]
enabled  = true
port     = 12345
filter   = classcord
logpath  = /var/log/classcord/classcord.log
maxretry = 3
bantime  = 600

# /etc/fail2ban/filter.d/classcord.conf
[Definition]
failregex = .*Tentative.*depuis <HOST>
ignoreregex =
```

```
sudo systemctl restart fail2ban
sudo fail2ban-client status classcord
```



Capture 03 — Édition de /etc/fail2ban/jail.local



Capture 04 — Service fail2ban actif (*Server ready*)

Pare-feu ufw

Seul le port du service est ouvert : `sudo ufw allow 12345/tcp`. Les autres ports entrants restent fermés une fois ufw activé.

Point de vigilance
Le filtre fail2ban doit correspondre exactement à ce que le serveur écrit dans son journal. Ce point est détaillé dans la section « Limites et pistes d'amélioration ».

7 Phase 5 — Sauvegarde automatique

Mercredi 18 juin 2025. Les comptes utilisateurs sont stockés dans `users.pkl` ; ce fichier est copié avec un horodatage.

Script sauvegarde.sh

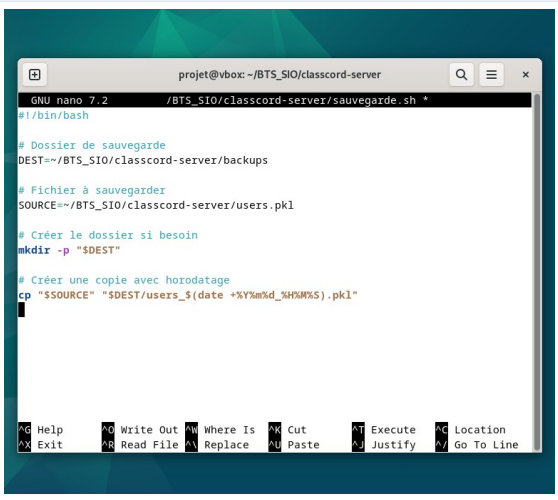
```
#!/bin/bash

# Dossier de destination
DEST=~/BTS_SIO/classcord-server/backups

# Fichier source à sauvegarder
SOURCE=~/BTS_SIO/classcord-server/users.pkl

# Création du dossier de sauvegarde si inexistant
mkdir -p "$DEST"

# Sauvegarde avec horodatage
cp "$SOURCE" "$DEST/users_$(date +%Y%m%d_%H%M%S).pkl"
```

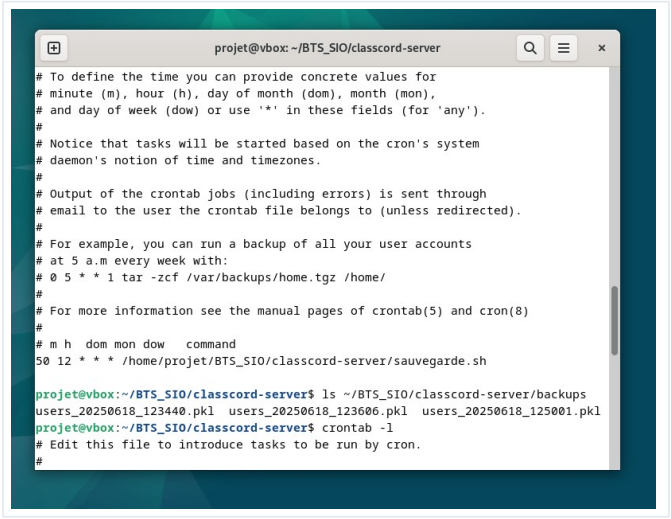


Capture 05 — Script sauvegarde.sh : copie horodatée de users.pkl

Planification avec cron

```
crontab -e
50 12 * * * /home/projet/BTS_SIO/classcord-server/sauvegarde.sh

crontab -l                          # vérification de la tâche
ls ~/BTS_SIO/classcord-server/backups
```



Capture 06 — Tâche cron et fichiers de sauvegarde horodatés dans backups/

> **Preuve d'exécution**
> Le fichier `users_20250618_125001.pkl` porte l'horodatage 12:50:01 : il a été créé par la tâche cron planifiée à 12h50.

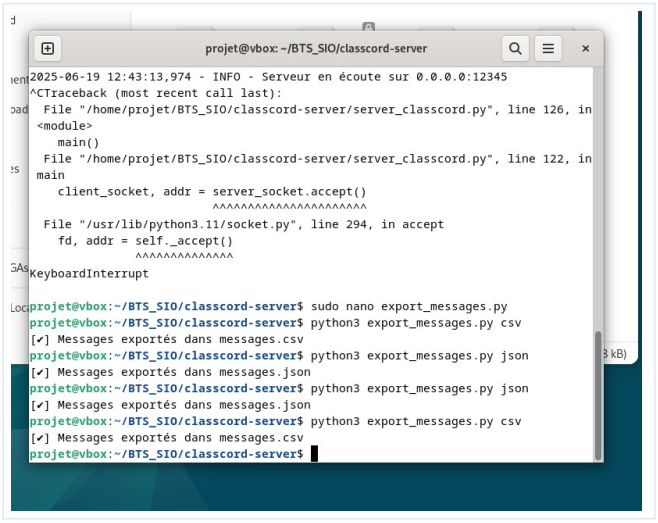
8 Phase 6 — Évolutions fonctionnelles

Jeudi 19 juin 2025. Le serveur est enrichi côté fonctionnalités, avec un client en ligne de commande pour les tests.

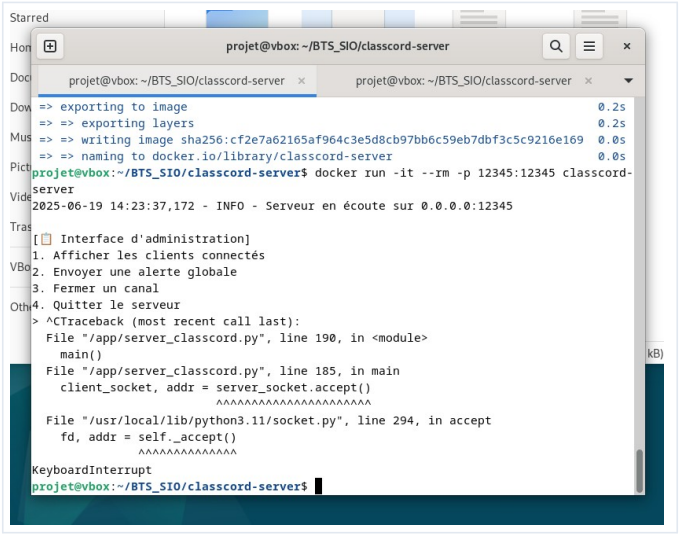
Fonctionnalité	Détail
Comptes	Inscription (`register`) et connexion (`login`), gestion des erreurs : utilisateur déjà existant, mot de passe incorrect
Canaux	Plusieurs canaux (#général, #dev, #admin), diffusion limitée aux utilisateurs connectés au canal
Messages système	Fonction `send_system_message()` : arrivée, départ, alertes du serveur
Base SQLite	Module `db.py` : tables `users` et `messages` (id, sender, content, timestamp, channel)
Export	`export_messages.py csv|json` : export des messages de la base
Administration	Menu console : clients connectés, alerte globale, fermeture d'un canal, arrêt du serveur
Client	`client_classcord.py` : interface en ligne de commande, échanges au format JSON

Format des messages

```
{
  "type": "message",
  "subtype": "global",
  "channel": "#dev",
  "from": "bob",
  "content": "Ping pour les devs !"
}
```



Capture 07 — Export des messages en CSV puis en JSON



Capture 08 — Serveur lancé dans Docker avec l'interface d'administration

9 Phase 7 — Finalisation

Vendredi 20 juin 2025. Regroupement du serveur complet et de sa documentation dans le dépôt GitHub (dossier « classcord complet + documentation »).

Livrable	Contenu
Serveur	`server_classcord.py`, `client_classcord.py`, `db.py`, `export_messages.py`
Exploitation	`start_server.sh`, `sauvegarde.sh`, service systemd
Docker	`Dockerfile` et `docker-compose.yml`
Documentation	Journaux de bord des jours 1 à 5, documentation de connexion, sécurité, fonctionnalités, Docker

10 Preuves relevées

Aucun jeu de tests formalisé n'a été conservé pour ce projet. Le tableau recense ce que les captures permettent de constater.

#	Vérification	Constat	Capture
1	Service systemd	*active (running)* et activé au démarrage	01
2	Image et conteneur Docker	Image construite, conteneur recréé et lancé	02
3	Service fail2ban	*active (running)*, *Server ready*	04
4	Sauvegarde planifiée	Tâche cron présente, fichiers horodatés dans backups/	06
5	Export des messages	Fichiers CSV et JSON générés sans erreur	07
6	Serveur dans Docker	Écoute sur 0.0.0.0:12345, interface d'administration disponible	08

11 Difficultés rencontrées

Problème	Cause	Solution
`sudo` refusé au départ	Utilisateur `projet` absent des sudoers	Ajout au groupe `sudo` avec `visudo`
IP partagée entre plusieurs machines	Plusieurs VM en NAT derrière la même adresse	Passage en mode pont (bridge) pour obtenir une IP locale propre
Connexion client impossible au départ	Paramètres hôte/port du client	Saisie explicite de l'hôte et du port 12345

12 Limites et pistes d'amélioration

Ce bilan s'appuie sur la relecture du code et des journaux du dépôt. Il sert de base pour une version 2 du service.

Constat	Piste d'amélioration
Le filtre fail2ban cherche « Tentative … depuis <IP> », mais le serveur n'écrit pas cette phrase : il journalise « Nouvelle connexion depuis (IP, port) ». Le chemin du journal diffère aussi (`server.log` dans le dossier du serveur).	Journaliser les échecs de connexion avec l'IP source, adapter le `failregex` et le `logpath`, puis valider par un test de bannissement (3 échecs).
Les comptes sont gérés dans `users.pkl` et non dans SQLite : `server_classcord.py` n'appelle pas encore `db.py`. Les exports générés sont donc vides.	Brancher `register_user()`, `login_user()` et `save_message()` dans le serveur, puis sauvegarder aussi `classcord.db`.
Les mots de passe sont stockés et journalisés en clair (le contenu brut d'un *register* apparaît dans le journal).	Hacher les mots de passe (bcrypt ou argon2) et ne plus journaliser les messages d'authentification.
Confusion possible entre l'IP de la VM et celle de l'hôte lors des tests externes.	Documenter l'IP de l'hôte et le port forwarding dans la documentation de connexion.
Service non redémarré après un reboot.	`systemctl enable classcord` et vérification au démarrage de la VM.

Constat	Piste d'amélioration
Documentation incomplète pour une reprise par un tiers.	README à jour et procédures courtes de dépannage (service arrêté, port fermé).

13 Compétences BTS SIO mobilisées

Ce projet mobilise les compétences suivantes du référentiel BTS SIO option SISR :

Bloc	Compétence	Activités réalisées
C1	Gérer le patrimoine informatique	Déploiement d'un service sur VM Debian, sauvegardes planifiées, documentation d'exploitation
C4	Travailler en mode projet	Planning sur 5 jours, un livrable et un push GitHub par jour, journal de bord
C5	Mettre à disposition des utilisateurs un service informatique	Service systemd, accès réseau pour les clients SLAM, conteneurisation Docker, protection ufw et fail2ban
C6	Organiser son développement professionnel	Documentation technique et journal de bord quotidien, mise en œuvre de Docker, fail2ban et SQLite

14 Sources

- Dépôt du projet — https://github.com/walo2428/classcord_server
- Projet d'origine — https://github.com/AstrowareConception/classcord-server
- Documentation Docker — https://docs.docker.com/
- Fail2ban — https://github.com/fail2ban/fail2ban/wiki
- systemd, pages de manuel — https://www.freedesktop.org/software/systemd/man/
- Python, module socket — https://docs.python.org/3/library/socket.html
- Wiki Debian — https://wiki.debian.org/